

7-3 · Embedding과 Vector DB · 정리·회고 0.5H

# 정리·회고

명령 아홉 줄을 두 장으로, 7-4 준비까지

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

# 30분 뒤에는

---

- ① 임베딩·Qdrant 명령 아홉 개를 「하려는 일」로 정리한다
- ② 오늘의 체크리스트 14개를 스스로 점검한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ qdrant·minio 두 컨테이너가 Up 인지 확인해 7-4를 준비한다

유사도 몇 점이 「관련 있음」인지는 절대 기준이 없다 — 우리 데이터로 재 보고 정한다

# 임베딩과 컬렉션 — 하려는 일과 코드

| 하려는 일        | 코드                                                                       |
|--------------|--------------------------------------------------------------------------|
| 임베딩 하나       | <code>emb.embed_query(text)</code>                                       |
| 임베딩 여러 개(권장) | <code>emb.embed_documents([t1, t2, ...])</code>                          |
| 접속           | <code>QdrantClient(url="http://localhost:6333")</code>                   |
| 컬렉션 생성       | <code>create_collection(name,<br/>VectorParams(size=, distance=))</code> |
| 존재 확인        | <code>client.collection_exists(name)</code>                              |

DIM = len(v1) — 컬렉션의 size 는 임베딩 모델의 차원과 반드시 같아야 한다

# 적재와 검색 — 하려는 일과 코드

| 하려는 일 | 코드                                                                                                 |
|-------|----------------------------------------------------------------------------------------------------|
| 적재    | <code>client.upsert(name, points=[...])</code><br><code>PointStruct(id=, vector=, payload=)</code> |
| 개수    | <code>client.count(name)</code>                                                                    |
| 검색    | <code>client.query_points(name,</code><br><code>query=vec, limit=).points</code>                   |
| 필터 검색 | <code>query_filter=Filter(</code><br><code>must=[FieldCondition(...)])</code>                      |

query\_points 의 결과는 .points 로 꺼낸다 — h.score 와 h.payload 를 함께 읽는다

# 임베딩 — 일곱 개를 스스로 확인해 보세요

---

- 1 임베딩이 무엇을 하는지 한 문장으로 말할 수 있다
- 2 임베딩 벡터의 차원 수를 코드로 확인했다
- 3 같은 모델로 만든 벡터끼리만 비교할 수 있다는 것을 안다
- 4 코사인 유사도를 직접 계산해 비슷한 쌍과 다른 쌍을 비교했다
- 5 문장 5개의 유사도 표를 만들어 예상과 대조했다
- 6 `embed_documents(배치)`가 왜 빠른지 실측으로 확인했다
- 7 부정문·식별자에서 임베딩이 약하다는 것을 직접 봤다

# Qdrant — 일곱 개를 스스로 확인해 보세요

|    |                                           |
|----|-------------------------------------------|
| 8  | Qdrant 를 볼륨과 함께 띄우고 대시보드를 열었다             |
| 9  | size 와 distance 를 지정해 컬렉션을 만들었다           |
| 10 | 차원을 일부러 틀려 Vector dimension error 를 직접 봤다 |
| 11 | payload 에 원문·분류·출처를 담아 적재했다               |
| 12 | 단어가 겹치지 않는 질문으로 의미 검색이 되는 것을 확인했다         |
| 13 | payload 필터를 걸어 검색 범위를 좁혀 봤다               |
| 14 | 7-2에서 만든 조각 전체를 임베딩해 적재하고 검색했다            |

# 증상으로 찾는 원인과 해결

| 증상                                  | 원인                                  | 해결                                                |
|-------------------------------------|-------------------------------------|---------------------------------------------------|
| 400 · Vector dimension error        | 컬렉션 size ≠ 모델 차원                    | len(벡터)로 확인 → 올바른 size 로 재생성·재적재                  |
| Connection refused (localhost:6333) | qdrant 가 안 떠 있거나 포트 매핑 누락           | docker ps 로 Up·6333 확인<br>없으면 docker start qdrant |
| create_collection 409 Conflict      | 같은 이름의 컬렉션이 이미 있다                   | collection_exists() 로 확인 후 생성 · recreate 신중히      |
| 검색 결과가 엉뚱하다                         | ① 다른 모델 ② text·vector 짝 어긋남 ③ 조각이 큼 | payload 원문 출력해 짝 확인<br>→ 그다음 chunk_size 조정        |

임베딩 모델을 바꾸면 dimension error 는 반드시 난다 — 컬렉션을 새로 만들고 전량 재적재

# 자주 나오는 질문 네 가지

---

- 1 유사도 몇 점 이상이면 관련 있다고 보나요?**  
절대 기준이 없다 — 모델·문서마다 분포가 달라 우리 데이터로 몇 개 재 보고 정한다
- 2 벡터DB 없이 파이썬 리스트로는 안 되나요?**  
수천 건까지는 된다 — 그 위로는 속도와 메모리 때문에 전용 저장소가 필요
- 3 임베딩 비용이 얼마나 드나요?**  
생성 모델보다 훨씬 싸다 — 다시 넣을 때마다 드니 이미 넣은 것은 다시 임베딩하지 않게 관리
- 4 로컬 모델(Ollama)로도 충분한가요?**  
학습·소규모에는 충분 — 한국어 품질은 모델마다 차이가 크니 우리 문서로 비교해 정한다

다음 차시(7-4)를 위한 준비

# 두 컨테이너 Up — qdrant 와 minio

|        |                                                        |
|--------|--------------------------------------------------------|
| Qdrant | <code>docker ps --filter name=qdrant</code> (계속 띄워 둔다) |
| MinIO  | <code>docker ps -a --filter name=minio</code>          |
| 없다면    | 4-1의 <code>docker run</code> 을 다시 실행                   |
| 패키지    | <code>pip install minio</code>                         |
| 확인     | 둘 다 Up · 4주차에 만든 버킷이 남아 있는지                            |

심화(선택): `chunk_size 200·1000` 두 벌 비교 · `Distance.EUCLID` 컬렉션 · `quotes` 데이터 주제 검색 · `client.scroll()`

다음 회차 예고

## 7-4 · MinIO·Qdrant 연동 — 창고와 색인 카드함

01

### 이론 1H

창고(MinIO)와  
카드함(Qdrant)

02

### 실습 1H

MinIO 버킷 구성과  
문서 적재

03

### 실습 1.5H

버킷 문서 순회  
임베딩→업서트

04

### 정리 0.5H

오늘 파이프라인  
정리·회고

준비: Qdrant 는 그대로 · 4·6주차 MinIO 가 떠 있으면 9000·9001 충돌 — docker compose stop 으로 먼저 내린다 · 임베딩은 같은 모델(같은 차원)

7-3 · 정리·회고 0.5H 마무리

## 다음 회차는 7-4 MinIO·Qdrant 연동

스토리지에 쌓인 문서를 자동으로 훑어 자르고 임베딩해 Qdrant 에 넣는 파이프라인을 만듭니다

✓ 원본 보관(MinIO)·검색 색인(Qdrant) 역할 분담

✓ Python 클라이언트로 버킷·객체 다루기

✓ Chunking → 임베딩 → 업서트 파이프라인

✓ payload 의 source 로 원본 역참조

준비할 것

qdrant·minio 두 컨테이너 Up